Assignment 2
Chess Endgame Outcome Classification

Dataset: UCI Chess (King-Rook vs. King-Pawn), KRKPA7

Name: Viradiya Tushar Rajeshbhai

BITS ID: 2026aj05073

Prepared: 29 October 2026

1. GitHub repository

https://github.com/tusharviradiya/aiml-ml-classification-assignment-2

Source code, dependencies, test data, five saved pipelines, and evaluation results.

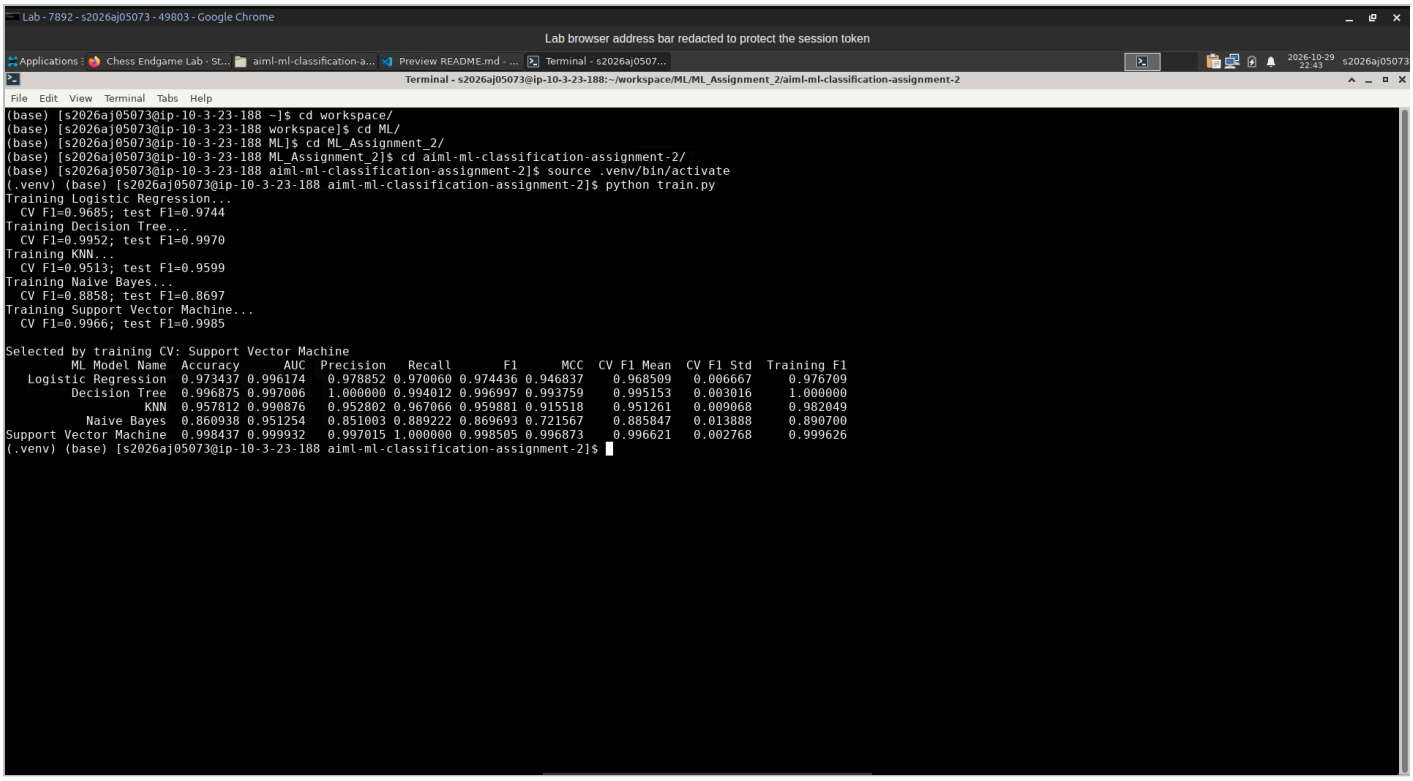
2. Live Streamlit application

https://chess-endgame-lab-tushar.streamlit.app/

Interactive CSV evaluation, model selection, six metrics, and model comparisons.

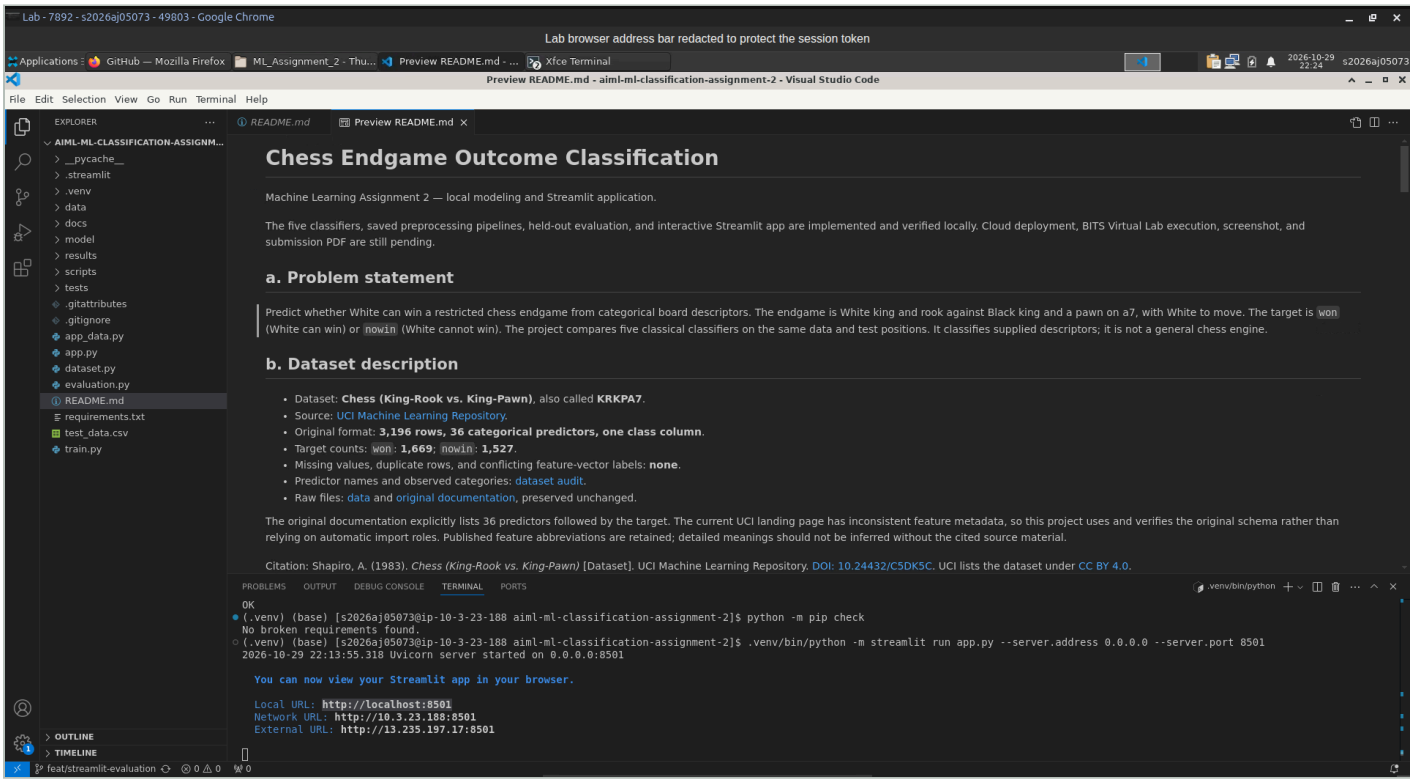
3. BITS Virtual Lab execution evidence

Primary evidence — completed model training



29 October 2026, 22:44. The lab terminal shows completed training, all five models and their metrics, and SVM selection.

Additional execution view — project and Streamlit server

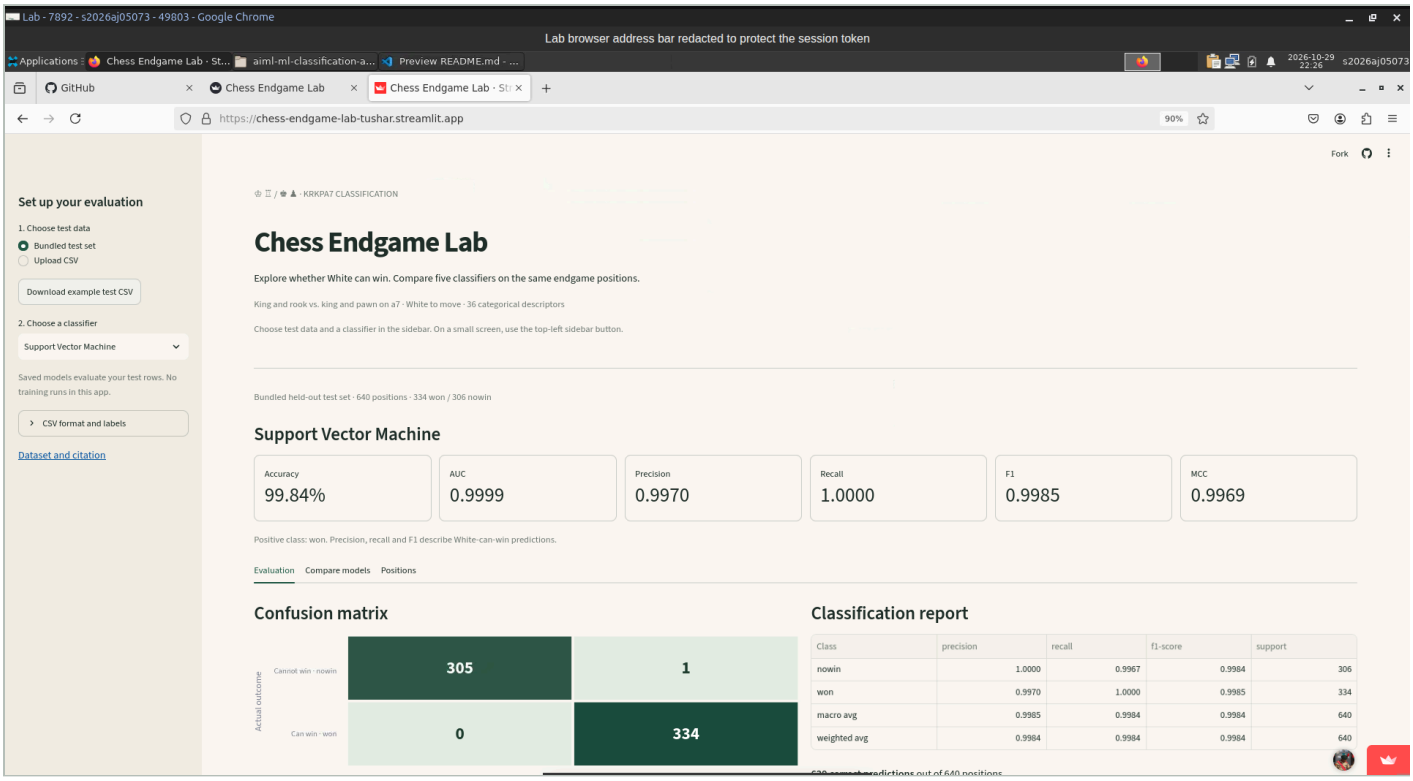


29 October 2026, 22:24. Project files, successful dependency check, and running Streamlit server. Browser address bars are redacted to protect the session token.

Run comparison: lab KNN test F1 is 0.9599; the committed local artifacts report 0.9661. This difference needs reconciliation; the README table reports the local artifacts.

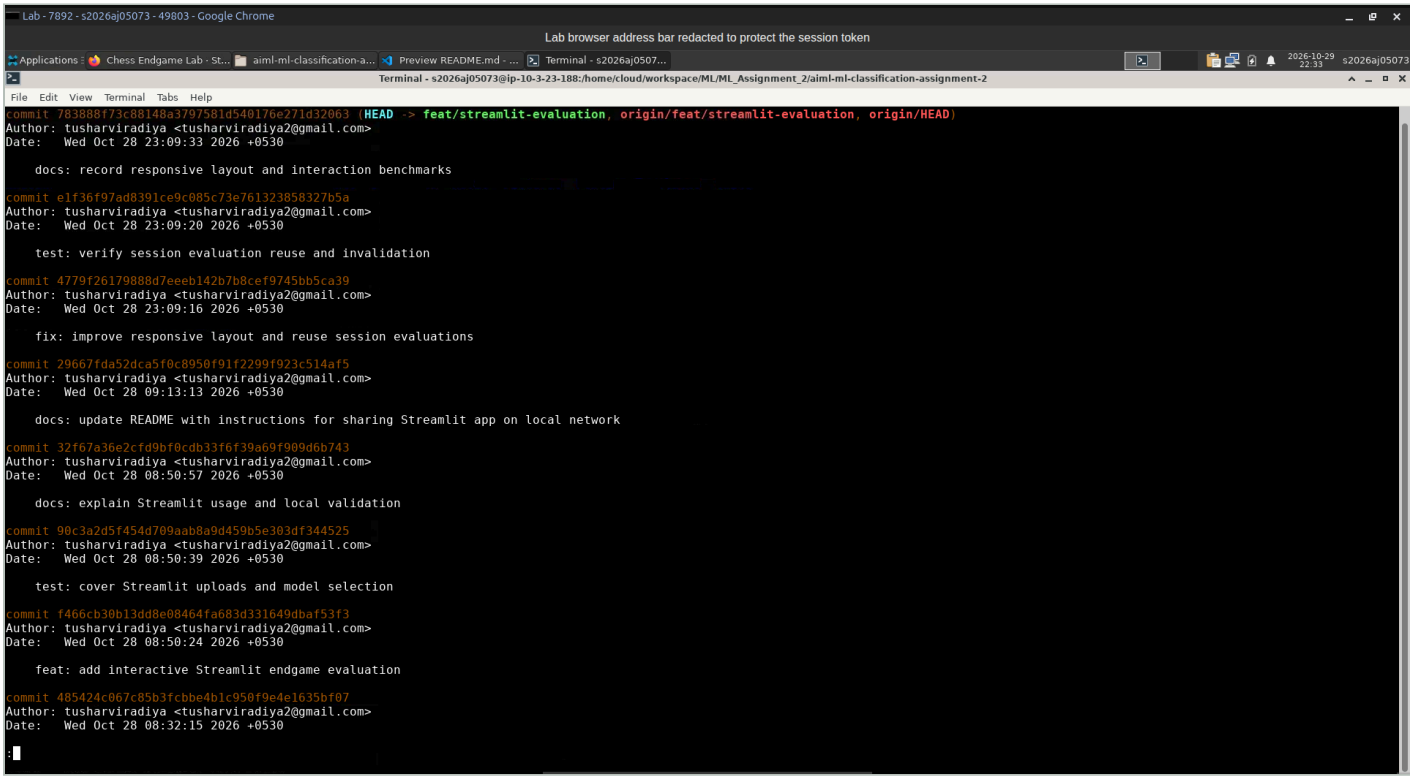
3. Supporting lab-session views

Public Streamlit Community Cloud application



29 October 2026, 22:26. The public cloud app opened from the lab browser. This view demonstrates the cloud interface, not execution of the lab-hosted app.

Repository commit history in the lab terminal



29 October 2026, 22:33. Repository history inspected inside the lab session. Supporting context only; completed training is documented on the preceding page.

All four supplied screenshots are included. Only the outer browser address bars are redacted; the captured execution output is preserved.

Chess Endgame Outcome Classification

Machine Learning Assignment 2 — local modeling and Streamlit application.

Open Chess Endgame Lab on Streamlit Community Cloud

The five classifiers, saved preprocessing pipelines, held-out evaluation, and interactive Streamlit app are implemented and verified locally. The app is deployed on Streamlit Community Cloud. BITS Virtual Lab project execution evidence was captured on 29 October 2026. The submission PDF is available in the repository root as `ML_Assignment_2_Submission.pdf`.

a. Problem statement

Predict whether White can win a restricted chess endgame from categorical board descriptors. The endgame is White king and rook against Black king and a pawn on a7, with White to move. The target is `won` (White can win) or `nowin` (White cannot win). The project compares five classical classifiers on the same data and test positions. It classifies supplied descriptors; it is not a general chess engine.

b. Dataset description

- Dataset: **Chess (King-Rook vs. King-Pawn)**, also called **KRKPA7**.
- Source: UCI Machine Learning Repository.
- Original format: **3,196 rows, 36 categorical predictors, one class column**.
- Target counts: `won`: **1,669**; `nowin`: **1,527**.
- Missing values, duplicate rows, and conflicting feature-vector labels: **none**.
- Predictor names and observed categories: dataset audit.
- Raw files: data and original documentation, preserved unchanged.

The original documentation explicitly lists 36 predictors followed by the target. The current UCI landing page has inconsistent feature metadata, so this project uses and verifies the original schema rather than relying on automatic import roles. Published feature abbreviations are retained; detailed meanings should not be inferred without the cited source material.

Citation: Shapiro, A. (1983). *Chess (King-Rook vs. King-Pawn)* [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C5DK5C. UCI lists the dataset under CC BY 4.0.

c. GitHub Repository Link

tusharviradiya/aiml-ml-classification-assignment-2

The repository records the modeling work in separate commits for environment setup, dataset auditing, implementation, saved artifacts, tests, and documentation. See the Git workflow for future development steps.

d. Models used

Logistic Regression, Decision Tree Classifier, K-Nearest Neighbor Classifier, Multinomial Naive Bayes, and RBF Support Vector Machine.

Evaluation procedure

1. Read all 36 predictors as categorical values. Name the final target column `outcome`; map `nowin` to 0 and `won` to 1.
2. Make one stratified 80/20 split with seed 42: **2,556 training positions** and **640 held-out positions**. Keep the target in the exported CSV files.
3. Fit full one-hot encoding inside each training pipeline. The 0/1 indicators need no additional numeric scaling. All classifiers use the same raw inputs.
4. Choose settings by five-fold stratified cross-validation **within training data only**, maximizing positive-class F1. `train.py` contains the exact grids; all candidate results are saved under `results/*_cv.csv`.
5. Refit the selected settings on all training rows and save each complete pipeline under `model/`. The overall classifier is selected by training CV F1.
6. Evaluate the saved pipelines on `test_data.csv`, without training on those rows.

One category, `spcop=t`, appears only once. An encoder trained on a fold may not see it; valid unseen categories map to an all-zero block for that field. `evaluation.py` separately rejects invalid codes using the audited category domains. It also checks columns, missing values, and target labels. It returns an unavailable AUC with an explanation if an uploaded subset contains one class.

Multinomial Naive Bayes treats each one-hot field as one category occurrence. This preserves nonnegative count inputs and meets the assignment's allowed Naive Bayes variant. Its independence assumptions are a limitation worth discussing. SVM uses decision values for AUC; they are not probabilities.

Comparison and observations

This section is regenerated by training. Review the observations and develop your own explanation of the results before submitting the assignment.

Generated by `python train.py`.

Training rows: 2556. Test rows: 640. Seed: 42. Positive class: won. Settings selected by five-fold training CV F1.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9734	0.9962	0.9789	0.9701	0.9744	0.9468
Decision Tree	0.9969	0.9970	1.0000	0.9940	0.9970	0.9938
KNN	0.9641	0.9939	0.9507	0.9820	0.9661	0.9284
Naive Bayes	0.8609	0.9513	0.8510	0.8892	0.8697	0.7216
Support Vector Machine	0.9984	0.9999	0.9970	1.0000	0.9985	0.9969

Model	CV F1 mean	CV F1 std	Training F1	Test F1
Logistic Regression	0.9685	0.0067	0.9767	0.9744
Decision Tree	0.9952	0.0030	1.0000	0.9970
KNN	0.9494	0.0093	1.0000	0.9661
Naive Bayes	0.8858	0.0139	0.8907	0.8697
Support Vector Machine	0.9966	0.0028	0.9996	0.9985

Selected model: **Support Vector Machine**, based on training CV F1.

Precision, recall and F1 use won as the positive class. AUC uses positive-class probabilities, or decision scores for SVM. MCC uses predicted labels. Precision/recall/F1 are set to zero when their denominator is zero.

SVM scores are decision values, not probabilities. Confusion matrices in evaluation.json use row=true, column=predicted, with order [nowin, won].

These are results for one held-out split of this restricted endgame dataset. They do not establish performance on arbitrary chess positions.

Observations

ML Model Name	Observation about model performance
Logistic Regression	Test F1 0.9744; 7 false wins and 10 missed wins. Training F1 0.9767. An additive linear baseline over encoded categories; it does not explicitly represent feature interactions.
Decision Tree	Test F1 0.9970; 0 false wins and 2 missed wins. Training F1 1.0000. Can learn conjunctions of feature conditions. Compare training and validation results when assessing overfitting.
KNN	Test F1 0.9661; 17 false wins and 6 missed wins. Training F1 1.0000. Distance-weighted training predictions can be perfect because a training row is its own neighbor; cross-validation is the more useful check.
Naive Bayes	Test F1 0.8697; 52 false wins and 37 missed wins. Training F1 0.8907. Models indicators as category counts under conditional independence. Dependencies among board descriptors are a possible limitation, not a proven cause of errors.
Support Vector Machine	Test F1 0.9985; 1 false wins and 0 missed wins. Training F1 0.9996. The RBF kernel permits nonlinear feature interactions. Its measured performance applies to this restricted endgame domain.
Overall winner for this dataset	**Support Vector Machine**, selected using the highest mean training cross-validation F1, before test-set comparison.

A false win means predicting won for a nowin position; a missed win means predicting nowin for a won position. Small score differences on one split do not establish statistical superiority. These observations should be reviewed and developed in the student's own words before submission.

Reproduce the local modeling workflow

The installed versions are pinned in `requirements.txt`. They require Python 3.12 or newer; this run was verified on **Python 3.14.4**. Use an isolated environment and the same pinned dependencies when loading saved models.

Run these commands from the repository root on Linux or BITS Virtual Lab:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python scripts/audit_dataset.py
python train.py
python -m unittest discover -s tests -v
python -m pip check
```

Training overwrites generated models, split CSVs, results, and the marked README section. Keep the seed and original input files unchanged to reproduce the split. Numerical results can vary slightly across platforms; the exact source indices, checksums, package versions, selected settings, and run time are recorded in `results/manifest.json` and `results/evaluation.json`.

To inspect a saved model without retraining:

```
import joblib
import pandas as pd
from evaluation import evaluate_model

model = joblib.load("model/svm.joblib")
test = pd.read_csv("test_data.csv", dtype=str)
result = evaluate_model(model, test)
print(result["metrics"])
print(result["confusion_matrix"])
```

The expected CSV schema is the 36 original feature columns plus `outcome`. Column order may vary; the evaluator restores model order. Target values must be `won` or `nowin`. Use `test_data.csv` as the example upload.

Run Chess Endgame Lab locally

After installing the pinned requirements, run:

```
.venv/bin/python -m streamlit run app.py
```

if want to run app and share url to same lan devcies run like this:

```
.venv/bin/python -m streamlit run app.py --server.address 0.0.0.0 --server.port 8501
```

Open **http://localhost:8501** in your browser. The app loads the saved models; starting it does not require a training run.

1. Start with **Bundled test set** to evaluate the committed 640 held-out positions.
2. Use the classifier dropdown to inspect any of the five saved models.
3. View all six metrics, the confusion matrix, classification report, and ROC curve.
4. Open **Compare models** for metrics calculated on the current test data and a downloadable comparison CSV.
5. Download the example CSV, select **Upload CSV**, and upload that file to verify the labeled-data evaluation path. Uploaded files must be UTF-8 CSV, no larger than 2 MB, with at most 5,000 test positions.
6. Open **Positions** to filter errors and download the selected model's predictions. Test row numbers refer to data rows in the current CSV.

Uploads are processed in memory and are not written to the repository. The current evaluation is retained only in that visitor's session so dropdown and filter changes can reuse it. New CSV content or a changed saved model triggers fresh evaluation; clearing an upload discards the session result. Only the saved classifiers use Streamlit's shared cache. No fitting or tuning occurs in the app. Invalid columns, duplicate headers, malformed rows, unsupported categories and labels produce readable errors. With one target class, AUC displays **N/A** with an explanation; the other metrics and 2×2 confusion matrix remain available.

Theme and upload limits are configured in `.streamlit/config.toml`. No secrets are needed to run this app. Streamlit's usage telemetry is disabled.

Local validation

The 16 automated checks cover saved-model reproducibility, split integrity, CSV parsing, all five dropdown choices, live upload evaluation without fitting, invalid uploads, clearing and replacing a file, error filtering, single-class data, missing saved models, and session-result reuse, isolation and invalidation. They use Streamlit's `AppTest` API to exercise the interface. The local server health endpoint was also checked successfully. The

responsive layout was tested in headless Chrome at phone, tablet, laptop and desktop sizes; see the responsiveness report for measurements and review steps.

```
.venv/bin/python -m unittest discover -s tests -v
.venv/bin/python -m pip check
```

Live Streamlit Community Cloud URL: Chess Endgame Lab.

A signed-out Chrome check on 28 October 2026 confirmed that the public app loads the 640-position bundled test set and displays the expected six SVM metrics, including 99.84% accuracy, with its confusion matrix and classification report.

Files and generated artifacts

Path	Purpose
`app.py`	Streamlit upload, model selection, metrics, charts and downloads
`app_data.py`	Strict CSV parsing and saved-model inference for the app
`.streamlit/config.toml`	Visual theme, upload limit and telemetry setting
`scripts/audit_dataset.py`	Standard-library audit of the original data
`dataset.py`	Original feature order, label mapping, category validation domains
`train.py`	Reproducible split, training CV, model saving, and evaluation
`evaluation.py`	Shared validation, inference, six metrics, confusion matrix and report
`data/train_data.csv`	Training positions with target
`test_data.csv`	Held-out positions with target; bundled app data and example upload
`model/*.joblib`	All five trained classifiers with fitted encoders
`results/metrics.csv`	Six test metrics plus training/CV F1
`results/evaluation.json`	Parameters, metrics, confusion matrices and reports
`results/test_predictions.csv`	Per-model predictions and scores on the held-out rows
`results/manifest.json`	Reproduction details and original zero-based row indices
`tests/test_workflow.py`	Saved-artifact, split-integrity and inference validation checks
`tests/test_app.py`	CSV parser and Streamlit interaction checks

BITS Virtual Lab execution evidence

The screenshots captured on **29 October 2026** document:

- Primary execution evidence: completed training, showing `python train.py`, all five classifiers, evaluation metrics, and SVM selection.
- Project and server execution, including a successful dependency check and the Streamlit server running on port 8501.
- Supporting cloud application view, showing the public Community Cloud app opened from the lab browser; this is not a lab-hosted app view.

- Supporting Git history view, showing repository history inside the lab terminal; this is not training evidence.

The outer browser address bars are redacted to protect the lab session token. Execution output is preserved. The training screenshot reports KNN test F1 **0.9599**, compared with **0.9661** in the local results table above. The cause has not been established; compare the lab code revision, dependencies, and selected KNN parameters before claiming exact reproduction. The README metrics table continues to describe the committed local artifacts.

Submission package and final checks

`ML_Assignment_2_Submission.pdf` contains the clickable GitHub link, clickable live app link, lab execution evidence, and this README content, in the assignment's required order. The completed training screenshot is the primary evidence for the requested lab screenshot. Four supplied images are arranged two per A4 page, with additional views labeled as supporting evidence.

- Reconcile the KNN difference between the lab training output and local artifacts; retain the lab execution logs for verification.
- Review and explain the model observations in your own words.
- Commit and push the updated README, redacted evidence, and submission PDF.
- Verify evaluator access to both links and check the exported PDF before upload.
- Submit the final assignment by **8 November 2026, 23:59**.

The assignment allows AI for learning support. Understand, verify, and explain the implementation and observations in your own words. Creating the PDF does not submit it to the assignment portal; complete the portal's final submission step.